

# Decidability

From here onwards we will use Turing machines. But our focus is on algorithms.

Turing machine merely serves as a precise model for the definition of algorithm.

We will no longer consider low-level Turing machines with states, tapes and transition functions.

When we describe a Turing machine, there are three ways to do so.

(i) formal description

(ii) Implementation description

(We describe how Turing machine moves its head and the way it stores data.)

(iii) high-level description

We use English to describe an algorithm.

Input to a turing machine is always a string.

If we want to input an object to a turing machine, we have to encode the object as a string.

Example: let  $A$  be the language consisting of all strings representing undirected graphs that are connected.

$$A = \{ \langle \underline{G} \rangle \mid G \text{ is connected undirected graph} \}$$

we can describe a turing machine  $M$  that decides  $A$  using following high-level description.

$M = 1$  on input  $\langle G \rangle$ , the encoding of a graph  $G$ :

1. Select a node and mark it.
2. Repeat the following steps until no new nodes are marked:
  3. for each node in  $G$ , mark it if it is attached by an edge to an already marked node.
4. Scan all nodes of  $G$  to determine whether they are all marked.

If they are all marked, **accept**,  
Otherwise, **reject**.

Intuitive notion  
of  
algorithms

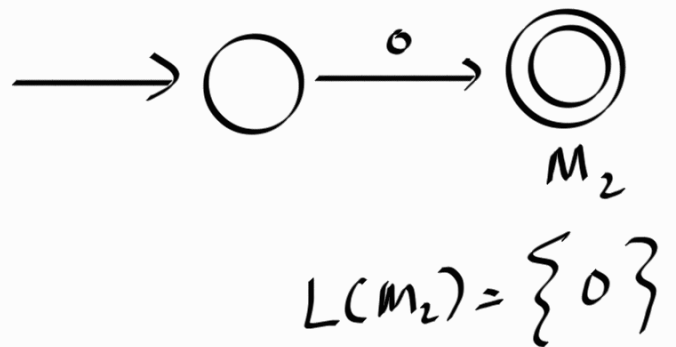
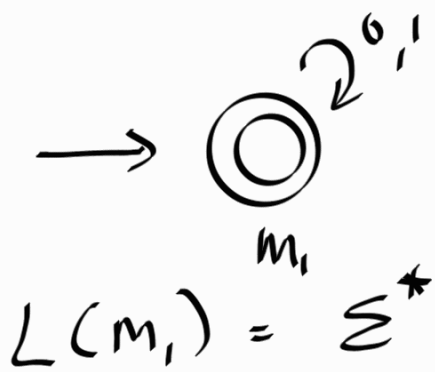
$\equiv$

Turing Machine  
algorithms

Decidable problems related to regular languages.

We choose to represent various computational problems by languages.

$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts input string } w \}$



$\langle M_1, "01" \rangle \in A_{DFA}$

$\langle M_1, "00000111" \rangle \in A_{DFA}$

$\langle M_2, "0" \rangle \in A_{DFA}$

$\langle M_2, "001" \rangle \notin A_{DFA}$

# Decidable problems on DFAs

$$A_{DFA} = \{ \langle B, w \rangle \mid B \text{ is a DFA that accepts } w \}$$

$$E_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}$$

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A, B \text{ are DFA and } L(A) = L(B) \}$$

## Theorem 4.1

$A_{DFA}$  is decidable.

we construct a TM  $M$  that decides  $A_{DFA}$ .

$M = \langle \langle B, w \rangle \rangle$ , where  $B$  is a DFA and  $w$  is a string.

1. Simulate  $B$  on  $w$ .
2. If simulation ends on an accept state, accept. If it ends on a non-accepting state, reject.

Using a similar argument, we can show that  $A_{NFA}$  and  $A_{REX}$  is decidable

$$A_{NFA} = \left\{ \langle N, w \rangle \mid \begin{array}{l} N \text{ is an NFA and} \\ N \text{ accepts } w \end{array} \right\}$$

$$A_{REX} = \left\{ \langle R, w \rangle \mid \begin{array}{l} R \text{ is a regular expression} \\ \text{that generates } w \end{array} \right\}$$

TM  $T$  that decides  $A_{NFA}$   $\rightarrow$

$T = \text{"on } \langle N, w \rangle, \text{ where } N \text{ is a NFA and } w \text{ is a string:"}$

1. Convert  $N$  to a DFA  $M$  using the procedure discussed in the class
2. Simulate  $M$  on  $w$ .
3. If simulation ends on an accept state, **accept**. If it ends on a non-accepting state, **reject**

## Theorem 4.4

$E_{DFA}$  is decidable

$$E_{DFA} = \{ \langle A \rangle \mid A \text{ is a DFA and } L(A) = \emptyset \}$$

We construct a Turing machine  $T$  that decides  $E_{DFA}$

$T =$  "on  $\langle A \rangle$ , where  $A$  is a DFA:

1. Mark the Start State of  $A$ .
2. Repeat until no new states are marked:
  3. Mark any state that has a transition coming into it from a state that is already marked.
4. If no accept states are marked, accept. otherwise, reject.

$$EQ_{DFA} = \{ \langle A, B \rangle \mid A, B \text{ are DFA, and } L(A) = L(B) \}$$

How about following TM  $K$ ?

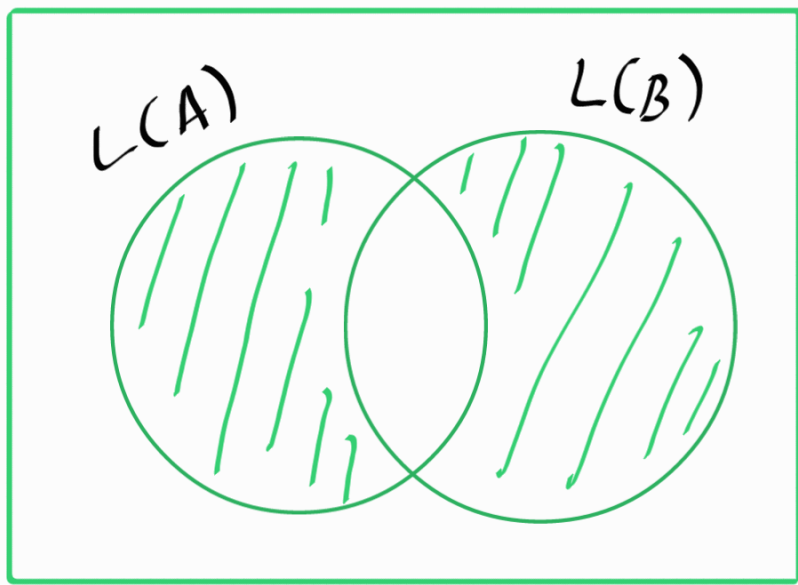
$K =$  "on  $\langle A, B \rangle$  where  $A, B$  are DFAs:  
1. Feed all strings into  $A$  and  $B$  one by one.  
If one machine accepts and other rejects,  
reject."

Is this a decider?

No

Is this a recognizer?

No



$$L(C) = (L(A) \cap \overline{L(B)}) \cup (L(B) \cap \overline{L(A)})$$

$$L(A) = L(B) \Rightarrow L(C) = \emptyset$$

$$L(C) = \emptyset \Rightarrow L(A) = L(B)$$

$$L(A) = L(B) \Leftrightarrow L(C) = \emptyset$$

Regular expressions are closed under  
 $\cup, \cap, -$  operators.

$L(C)$  is a regular language.

$\therefore$  we can create DFC  $C$  that recognizes  $L(C)$ .

Correct TM  $K$  is as follows

$K = \text{"on } \langle A, B \rangle$ , where  $A, B$  are DFAs,

1. Construct DFA  $C$  as mentioned earlier.
2. Run TM  $T$  from Theorem 4.4 on input  $\langle C \rangle$ .
3. If  $T$  accepts, accept; otherwise reject.

